

HW1 - STAT 701

Wells Guan

Date: September 16, 2026

Throughout, ACF denotes the autocovariance function. The appendix contains the Python source used to generate the figures.

Problem 1. Ex. 1.3.

Suppose that $m_t = c_0 + c_1 t + c_2 t^2$, $t = 0, \pm 1, \dots$,

(a) Show that

$$m_t = \sum_{i=-2}^2 a_i m_{t+i} = \sum_{i=-3}^3 b_i m_{t+i}, \quad t = 0, \pm 1, \dots$$

where $a_2 = a_{-2} = -\frac{3}{35}$, $a_1 = a_{-1} = \frac{12}{35}$, $a_0 = \frac{17}{35}$, and $b_3 = b_{-3} = -\frac{2}{21}$, $b_2 = b_{-2} = \frac{3}{21}$, $b_1 = b_{-1} = \frac{6}{21}$, $b_0 = \frac{7}{21}$.

(b) Suppose that $X_t = m_t + Z_t$, where $(Z_t)_{t \in \mathbb{Z}}$ is an independent sequence of normal random variables, each with mean 0 and variance σ^2 . Let

$$U_t = \sum_{i=-2}^2 a_i X_{t+i} \quad \text{and} \quad V_t = \sum_{i=-3}^3 b_i X_{t+i}$$

then

- (i) Find the means and variances of U_t and V_t .
- (ii) Find the correlations between U_t and U_{t+1} and between V_t and V_{t+1} .
- (iii) Which of the two filtered series $(U_t)_{t \in \mathbb{Z}}$ and $(V_t)_{t \in \mathbb{Z}}$ would you expect to be smoother in appearance?

Sol.

(a) A filter with weights w_i preserves every quadratic trend if

$$\sum_i w_i = 1, \quad \sum_i i w_i = 0, \quad \sum_i i^2 w_i = 0.$$

Indeed, expand $m_{t+i} = m_t + (c_1 + 2c_2 t)i + c_2 i^2$ and sum. For a_i , the three sums are $(17 + 24 - 6)/35 = 1$, 0 by symmetry, and $(24 - 24)/35 = 0$. For b_i , they are $(7 + 12 + 6 - 4)/21 = 1$, 0, and $(12 + 24 - 36)/21 = 0$. Thus both filters reproduce m_t .

(b)(i) Independence gives

$$\mathbb{E}U_t = \mathbb{E}V_t = m_t, \quad \text{Var}(U_t) = \sigma^2 \sum_i a_i^2 = \frac{17}{35}\sigma^2, \quad \text{Var}(V_t) = \sigma^2 \sum_i b_i^2 = \frac{1}{3}\sigma^2.$$

(ii) Only overlapping noise terms contribute, so

$$\text{Cov}(U_t, U_{t+1}) = \sigma^2 \sum_{i=-1}^2 a_i a_{i-1} = \frac{48}{175}\sigma^2, \quad \text{Cov}(V_t, V_{t+1}) = \sigma^2 \sum_{i=-2}^3 b_i b_{i-1} = \frac{12}{49}\sigma^2.$$

For $\sigma^2 > 0$, the requested correlations are

$$\text{Corr}(U_t, U_{t+1}) = \frac{48}{85}, \quad \text{Corr}(V_t, V_{t+1}) = \frac{36}{49}.$$

(iii) V_t should appear smoother: it has smaller noise variance and greater lag-one correlation. More directly, the variances of the noise increments are $74\sigma^2/175$ for U and $26\sigma^2/147$ for V . Both have the same deterministic trend. If $\sigma^2 = 0$, both equal m_t .

Problem 2. Ex. 1.7.

Let $Z_t, t = 0, \pm 1, \dots$, be independent normal random variables, each with mean 0 and variance σ^2 , and let a, b , and c be constants.

Which, if any, of the following processes are stationary? For each stationary process, specify the mean and autocovariance function.

(a)

$$X_t = a + bZ_t + cZ_{t-1}.$$

(b)

$$X_t = a + bZ_0.$$

(c)

$$X_t = Z_1 \cos(ct) + Z_2 \sin(ct).$$

(d)

$$X_t = Z_0 \cos(ct).$$

(e)

$$X_t = Z_t \cos(ct) + Z_{t-1} \sin(ct).$$

(f)

$$X_t = Z_t Z_{t-1}.$$

Sol.

All variables have finite second moments. Write $\delta_r(h) = \mathbf{1}_{\{h=r\}}$. Assume $\sigma^2 > 0$; if $\sigma^2 = 0$, all six processes are deterministic constants and hence stationary.

(a) The mean is a , and independence gives

$$\gamma_X(h) = \sigma^2[(b^2 + c^2)\delta_0(h) + bc\{\delta_1(h) + \delta_{-1}(h)\}].$$

Thus the process is stationary.

(b) The mean is a and $\gamma_X(h) = b^2\sigma^2$ for every h . The process is stationary, although it is constant across time within each realization.

(c) The mean is zero and

$$\gamma_X(t, t+h) = \sigma^2\{\cos(ct)\cos(c(t+h)) + \sin(ct)\sin(c(t+h))\} = \sigma^2\cos(ch).$$

Thus the process is stationary.

(d) The mean is zero, but $\text{Var}(X_t) = \sigma^2 \cos^2(ct)$. Since t is integer-valued, the process is stationary precisely when $c \in \pi\mathbb{Z}$. For $c = m\pi$, its autocovariance is $\gamma_X(h) = \sigma^2(-1)^{mh}$. Otherwise the variances at $t = 0, 1$ differ.

(e) The mean is zero and the variance is σ^2 , but

$$\text{Cov}(X_t, X_{t+1}) = \sigma^2 \sin(ct) \cos(c(t+1)).$$

This is zero at $t = 0$ and $-\sigma^2 \sin c$ at $t = -1$. Consequently stationarity requires $c \in \pi\mathbb{Z}$. This condition is also sufficient: for $c = m\pi$, $X_t = (-1)^{mt} Z_t$, with $\gamma_X(h) = \sigma^2 \delta_0(h)$.

(f) Independence gives $\mathbb{E}X_t = 0$ and $\text{Var}(X_t) = \mathbb{E}Z_t^2 \mathbb{E}Z_{t-1}^2 = \sigma^4$. For $|h| = 1$, a noise factor appears only once and has mean zero; for $|h| \geq 2$, the products use disjoint independent variables. Hence

$$\gamma_X(h) = \sigma^4 \delta_0(h),$$

and the process is stationary. Its adjacent observations need not be independent.

Problem 3. Ex. 1.13.

Let $\{S_t : t = 0, 1, 2, \dots\}$ be the random walk with constant drift μ , defined by

$$S_0 = 0, \quad \text{and} \quad S_t = \mu + S_{t-1} + X_t, \quad t = 1, 2, \dots,$$

where $X_1, X_2, \dots$ are i.i.d. random variables with mean 0 and variance σ^2 . Compute the mean of S_t and the autocovariance function of the process $\{S_t\}$. Show that $\{\nabla S_t\}$ is stationary and compute its mean and autocovariance function.

Sol.

Notice

$$\mathbb{E}S_t = \mu + \mathbb{E}S_{t-1}, \quad S_0 = 0$$

and hence $\mathbb{E}S_t = t\mu$ for $t \in \mathbb{N}$. We know

$$S_t = t\mu + \sum_{i=1}^t X_i$$

as well, then we know

$$\gamma_S(t, s) = \mathbb{E} \left(\sum_{i=1}^t X_i \right) \left(\sum_{i=1}^s X_i \right) = (t \wedge s) \sigma^2.$$

We know

$$\nabla S_t = S_t - S_{t-1} = \mu + X_t =: Y_t, \quad \mathbb{E}Y_t = \mu$$

and it is easy to check $Y_t \in L^2$ and notice

$$\gamma_Y(t, t+h) = \sigma^2 \delta_0(h)$$

which is constant about t and hence Y_t is a stationary process, with the autocovariance function above.

Problem 4. Ex. 1.17.

If

$$\mathbf{X} = (X_1, \dots, X_n)'$$

is a random vector with covariance matrix Σ , show that Σ is singular if and only if there exists a nonzero vector

$$\mathbf{b} = (b_1, \dots, b_n)' \in \mathbb{R}^n$$

such that

$$\text{Var}(\mathbf{b}'\mathbf{X}) = 0.$$

Sol.

For every vector b , $\text{Var}(b^T X) = b^T \Sigma b$. If Σ is singular, choose $b \neq 0$ in its kernel, giving zero variance. Conversely, write $\Sigma = Q\Lambda Q^T$, where Q is orthogonal and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ with $\lambda_i \geq 0$. If $b \neq 0$ and $b^T \Sigma b = 0$, then

$$0 = \sum_{i=1}^n \lambda_i (Q^T b)_i^2.$$

At least one component of $Q^T b$ is nonzero, and its corresponding eigenvalue must be zero. Therefore Σ is singular.

Problem 5. Sample Autocovariances

Say I give you a time series $(X_1, \dots, X_n)$ that you know is stationary. Prove that the sample autocovariance function

$$\hat{\gamma}(h) = n^{-1} \sum_{t=1}^{n-|h|} (X_{t+|h|} - \bar{X})(X_t - \bar{X})$$

for $|h| < n$ gives a positive semi-definite sample autocovariance matrix

$$\hat{\Sigma}_{j,k} = \hat{\gamma}(|j - k|).$$

Sol.

Notice that define

$$Y_t = \begin{cases} X_t - \bar{X}, & 1 \leq t \leq n \\ 0, & \text{else} \end{cases}$$

and then

$$n\hat{\gamma}(|j - k|) = \sum_{t \in \mathbb{Z}} Y_{t+j} Y_{t+k}.$$

For $\mathbf{a} = (a_1, \dots, a_n)^T \in \mathbb{R}^n$, we know

$$\mathbf{a}^T \hat{\Sigma} \mathbf{a} = \mathbf{a}^T \left(\sum_{j=1}^n a_j \hat{\gamma}(|j - i|) \right)_{i=1}^n = \sum_{i=1}^n \sum_{j=1}^n a_i \hat{\gamma}(|i - j|) a_j,$$

and

$$\begin{aligned} n \sum_{i=1}^n \sum_{j=1}^n a_i \hat{\gamma}(|i-j|) a_j &= \sum_{i=1}^n \sum_{j=1}^n \sum_{t \in \mathbb{Z}} a_i a_j Y_{t+i} Y_{t+j} \\ &= \sum_{t \in \mathbb{Z}} \left(\sum_{i=1}^n a_i Y_{t+i} \right)^2 \geq 0 \end{aligned}$$

and hence $\mathbf{a}^T \hat{\Sigma} \mathbf{a} \geq 0$.

Problem 6. Causality

Consider the AR(1) process

$$X_t = 2X_{t-1} + Z_t,$$

where Z_t is some white noise with variance σ^2 . Define

$$Z_t^* = \frac{1}{4}Z_t - \frac{3}{4} \sum_{j=1}^{\infty} 2^{-j} Z_{t+j}.$$

1. Prove that Z_t^* is a white noise. What is its variance?
2. Prove that

$$X_t = \frac{1}{2}X_{t-1} + Z_t^*.$$

Sol.

1. It is easy to check that $\mathbb{E}Z_t^* = 0$ and

$$\begin{aligned} \gamma_{Z^*}(t, t+h) &= \mathbb{E} \left(\frac{1}{4}Z_t - \frac{3}{4} \sum_{j=1}^{\infty} 2^{-j} Z_{t+j} \right) \left(\frac{1}{4}Z_{t+h} - \frac{3}{4} \sum_{j=1}^{\infty} 2^{-j} Z_{t+h+j} \right) \\ &= \begin{cases} \sigma^2 \left(\frac{1}{16} + \frac{9}{16} \sum_{j=1}^{\infty} 2^{-2j} \right), & h = 0 \\ \sigma^2 \left(-3 \cdot 2^{-|h|}/16 + \frac{9}{16} \sum_{j=1}^{\infty} 2^{-2j} 2^{-|h|} \right), & \text{else} \end{cases} \\ &= \begin{cases} \frac{\sigma^2}{4}, & h = 0 \\ 0, & \text{else} \end{cases}. \end{aligned}$$

Therefore, Z_t^* is a white noise with variance $\sigma^2/4$.

2. We use the stationary finite-variance solution of this AR equation. The infinite sums converge in L^2 because the coefficients are absolutely summable and the second moments are bounded. Since $Z_t = X_t - 2X_{t-1}$, we know

$$\begin{aligned} Z_t^* &= \frac{1}{4}(X_t - 2X_{t-1}) - \frac{3}{4} \sum_{j=1}^{\infty} 2^{-j} (X_{t+j} - 2X_{t+j-1}) \\ &= \frac{1}{4}(X_t - 2X_{t-1}) - \frac{3}{4} \left(\sum_{j=1}^{\infty} 2^{-j} X_{t+j} - \sum_{j=0}^{\infty} 2^{-j} X_{t+j} \right) \\ &= X_t - \frac{1}{2}X_{t-1}, \end{aligned}$$

For a truncation at $j = M$, the expression equals $X_t - \frac{1}{2}X_{t-1} - \frac{3}{4}2^{-M}X_{t+M}$. The last term converges to zero in L^2 for the stationary solution. Without a bounded-second-moment or stationarity assumption, an additional homogeneous solution $C2^t$ is possible and the claimed representation need not hold.

Problem 7. Data exploration

Please download and take a look at the CET time series from Canvas.

1. Plot the data in some coherent way. This includes using proper timestamps and labeling the axes with the appropriate units. Does the data look like a stationary process? Why or why not? Perform some exploratory analysis to support your conclusion.
2. Select a segment containing

$$n = 365$$

contiguous observations, with no gaps or missing values.

Estimate the sample autocovariance function (ACF) for increasingly large lags k , and plot the results.

Discuss what happens as k increases and explain what you think is causing this behavior.

3. Design a linear mean function for the data using a constant and a first-degree term. For example, construct the design matrix using the following pseudocode:

```
1 t = 1:365
2 X = [tj^k for tj in t, k in 0:1]
```

Fit the linear model

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

using this design matrix.

Plot the residuals

$$\hat{\boldsymbol{\varepsilon}} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}.$$

Comment on what you observe. Do the residuals look like a stationary process? If not, describe the structures that suggest nonstationarity. Repeat the previous exploratory analyses to support your conclusion.

- 4*. Expand the analysis to

$$n = 3 \cdot 365$$

observations.

Construct a new design matrix $\mathbf{X}$ containing:

- a constant term;
- a linear trend;
- the seasonal component

$$\cos\left(\frac{2\pi t}{365}\right);$$

- the seasonal component

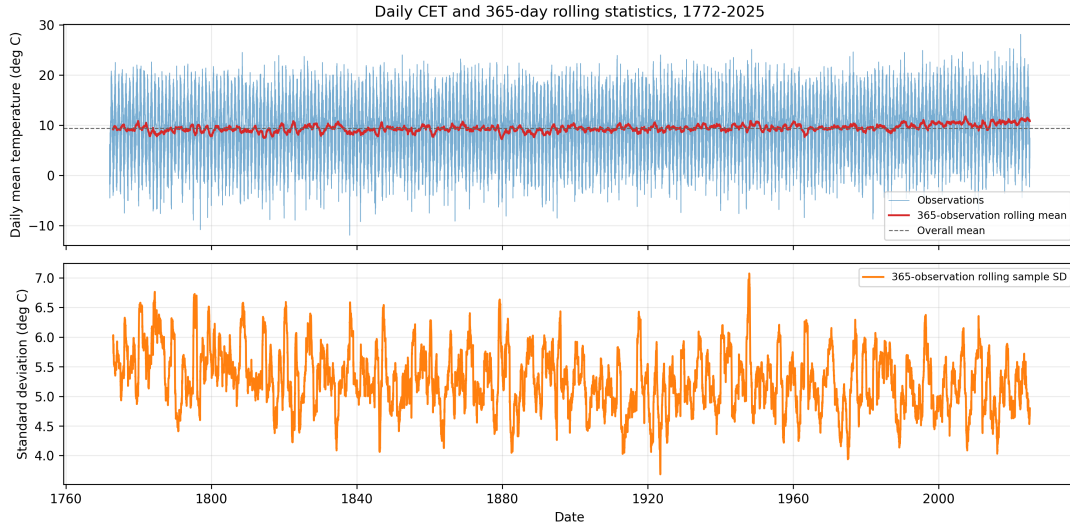
$$\sin\left(\frac{2\pi t}{365}\right).$$

Fit the least-squares mean function again and examine the residuals.

Normally, fitting a trigonometric function requires estimating both its frequency and its phase, which appears to be a nonlinear problem. Explain why the sine-and-cosine design-matrix approach correctly accounts for the phase using an ordinary linear least-squares model.

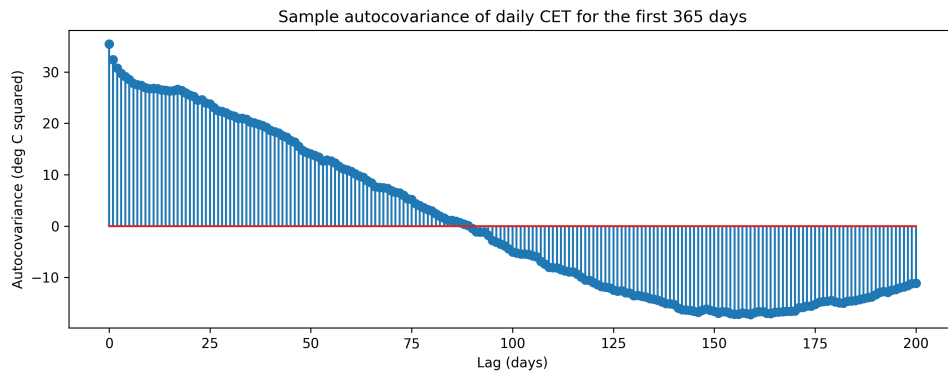
Sol.

1.



The raw series does not appear stationary. The displayed summaries use 365-day trailing windows, not monthly averages. They show changes in the long-run mean, particularly toward the end of the record. Separately, grouping daily observations by calendar month gives mean temperatures of about 3.48°C in January and 16.06°C in July. Seasonal means and long-run changes are inconsistent with a constant unconditional mean. Variation in rolling standard deviations alone is not conclusive evidence of nonstationarity.

2.



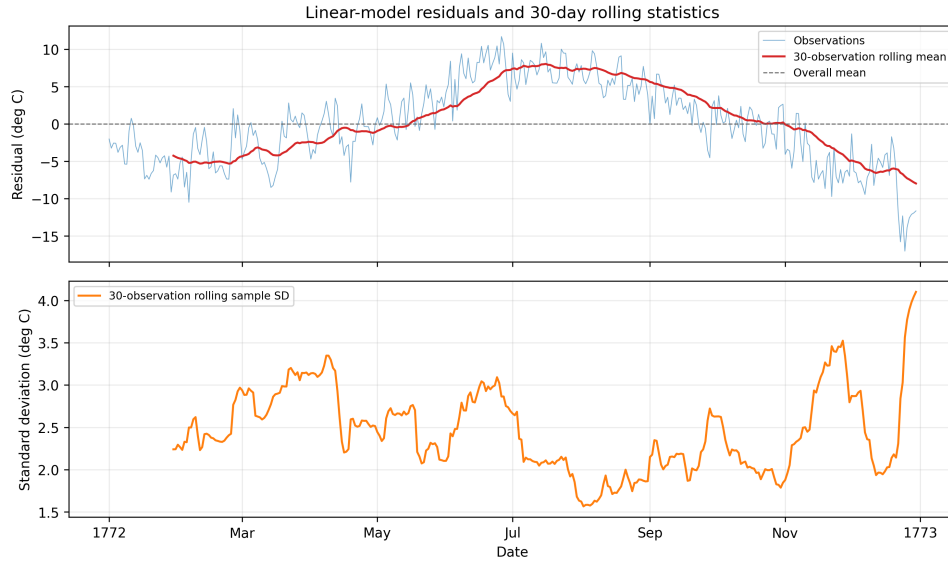
Throughout this assignment, ACF denotes the autocovariance function. We use

$$\hat{\gamma}(k) = \frac{1}{n} \sum_{t=1}^{n-k} (x_{t+k} - \bar{x})(x_t - \bar{x}), \quad 0 \leq k < n,$$

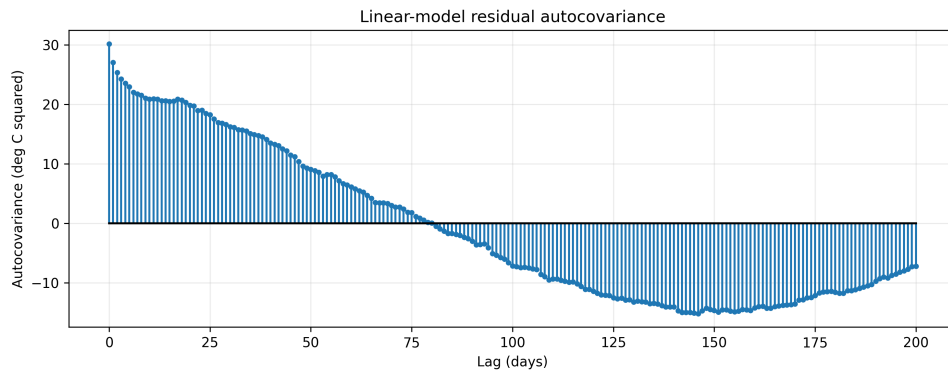
without dividing by $\hat{\gamma}(0)$. For these 365 daily observations, $\hat{\gamma}(0) \approx 35.47^\circ\text{C}^2$. The sample autocovari-

ance crosses zero near lag 90 and reaches a minimum of approximately -17.19°C^2 at lag 159, before rising again. This pattern is consistent with an annual seasonal cycle: nearby observations tend to have similar deviations from the mean, whereas observations roughly half a year apart often have deviations of opposite signs. At lag k , only $n - k$ pairs contribute, so large-lag estimates require caution.

3.



The first 365 observations are consecutive daily values starting on 1772-01-01, with no missing values. Using the first-degree specification, $\hat{m}_t = 5.1995 + 0.021813t$. The residuals retain a seasonal shape: negative near the beginning and end, and positive around the middle. Their 30-day rolling means range from about -7.98 to 8.05°C . The residual sample autocovariances at lags 0, 1, 30, 90 are approximately 30.18, 27.02, 16.23, -3.04 in squared degrees Celsius. These broad changes suggest that a straight line has not removed the seasonal mean. Autocovariance at nonzero lags alone does not imply nonstationarity, and the residual mean is zero by the OLS normal equations.

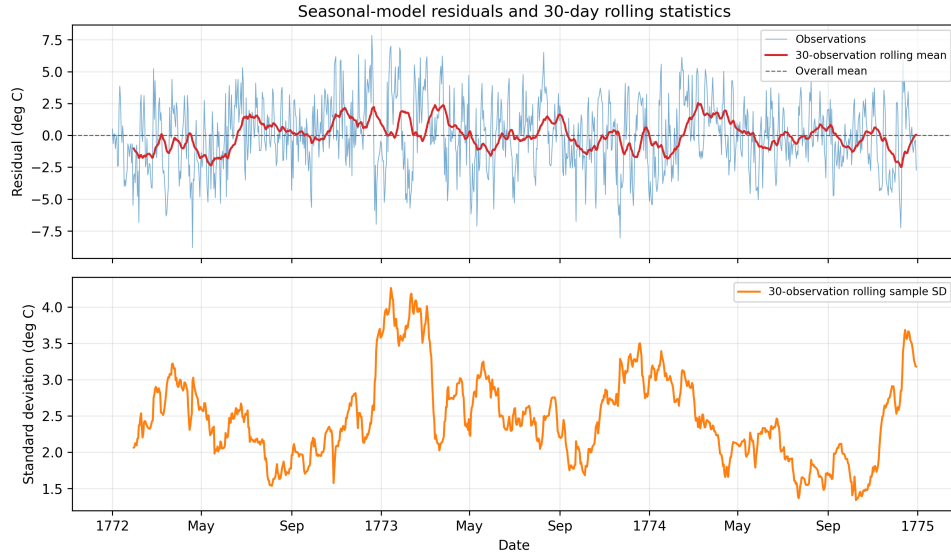


4.

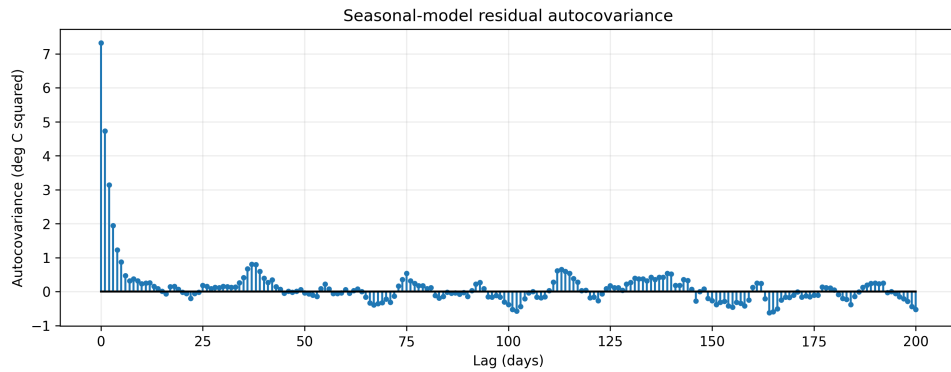
For the first 1095 observations, the fitted mean is

$$\hat{m}_t = 9.2522 - 0.000116t - 6.4981 \cos(2\pi t/365) - 2.8396 \sin(2\pi t/365).$$

The residual 30-day rolling mean ranges from about -2.48 to 2.54°C , and the rolling sample standard



deviation ranges from 1.34 to 4.26°C. The sample autocovariances at lags 0, 1, 30, 90 are approximately 7.33, 4.73, 0.15, −0.14 in squared degrees Celsius. The seasonal mean structure is substantially reduced, while short-term dependence remains. These plots are compatible with a closer approximation to stationarity, but do not establish it; the 30-day variability estimates also fluctuate. The sample length differs from part 3, so the numerical differences cannot all be attributed to the model. For fixed $\omega = 2\pi/365$,



any phase can be represented through

$$a \cos(\omega t) + b \sin(\omega t) = R \cos(\omega t - \phi), \quad R = \sqrt{a^2 + b^2}, \quad \phi = \text{atan2}(b, a).$$

Thus a, b are estimated by ordinary linear least squares. Only the frequency is fixed in advance; if it were unknown, estimating it would generally require a nonlinear search. When $R = 0$, the phase is immaterial.

Problem 8. Trigonometric expansions

- a. Let Z_1 and Z_2 be i.i.d. $\mathcal{N}(0, 1)$ variables, and let $\omega \in [-\pi, \pi)$ be a constant. Define the process

$$Y_t = Z_1 \cos(\omega t) + Z_2 \sin(\omega t).$$

Calculate its ACF. Is it stationary? Is it strictly stationary?

- b. Pick n to be your favorite even integer, say 512 (although 1024 is also a very good even integer).

Set

$$\omega_j = \frac{2\pi j}{n}, \quad j = 0, 1, \dots, n/2,$$

and simulate $(n/2 + 1)$ time series of length n given by

$$\begin{aligned} X_{0t} &= Z_0, & Z_0 &\sim \mathcal{N}(0, 1), \\ X_{jt} &= Z_{j1} \cos(\omega_j t) + Z_{j2} \sin(\omega_j t), & Z_{j1}, Z_{j2} &\sim \mathcal{N}(0, 2), \quad 1 \leq j < n/2, \\ X_{n/2,t} &= Z_{n/2}(-1)^t, & Z_{n/2} &\sim \mathcal{N}(0, 1). \end{aligned}$$

Now define a new process by

$$X_t = n^{-1/2} \sum_{j=0}^{n/2} X_{jt}.$$

1. Plot your data side-by-side with a white noise series of n i.i.d. $\mathcal{N}(0, 1)$ variables. What do you see?
2. Derive the ACF for X_t .
3. Play around with generalizing to

$$\text{Var}(Z_{j1}) = \text{Var}(Z_{j2}) = \sigma_j^2, \quad j \geq 1,$$

but otherwise not changing your code. Re-run everything and look at the resulting plot of your simulated process. Can you describe the effect of different choices for $\{\sigma_j^2\}$, for example, how things look if σ_j^2 increases or decreases with j ?

Sol.

a. We know

$$\begin{aligned} \gamma_Y(t, t+h) &= \mathbb{E}(Z_1 \cos(\omega t) + Z_2 \sin(\omega t))(Z_1 \cos(\omega(t+h)) + Z_2 \sin(\omega(t+h))) \\ &= \cos(\omega t) \cos(\omega(t+h)) + \sin(\omega t) \sin(\omega(t+h)) \\ &= \cos(\omega h), \end{aligned}$$

which is constant about t and hence Y_t is a stationary process, with autocovariance function

$$\gamma_Y(h) = \cos(\omega h).$$

For $\mathbf{t} := (t_1, \dots, t_k) \in \mathbb{Z}^k$, we know

$$\mathbf{Y}_{\mathbf{t}} := (Y_{t_1}, \dots, Y_{t_k})^T = \begin{bmatrix} \cos(\omega t_1) & \sin(\omega t_1) \\ \vdots & \vdots \\ \cos(\omega t_k) & \sin(\omega t_k) \end{bmatrix} \begin{bmatrix} Z_1 \\ Z_2 \end{bmatrix}$$

and hence $\mathbf{Y}_{\mathbf{t}}$ is a normal random vector with covariance matrix

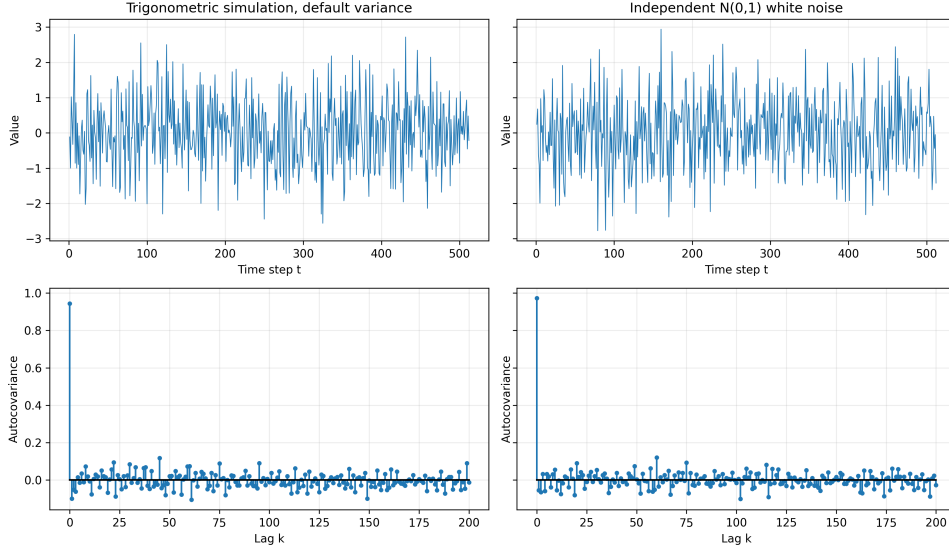
$$\begin{bmatrix} \cos(\omega t_1) & \sin(\omega t_1) \\ \vdots & \vdots \\ \cos(\omega t_k) & \sin(\omega t_k) \end{bmatrix} \begin{bmatrix} \cos(\omega t_1) & \sin(\omega t_1) \\ \vdots & \vdots \\ \cos(\omega t_k) & \sin(\omega t_k) \end{bmatrix}^T = [\cos(\omega(t_i - t_j))]_{i,j=1}^k$$

and hence

$$\mathbf{Y}_{\mathbf{t}} \stackrel{d}{=} \mathbf{Y}_{\mathbf{t}+h}$$

where $\mathbf{t} + h := (t_1 + h, \dots, t_k + h)$ for any $h \in \mathbb{Z}$ because they are Gaussian vectors with the same zero mean and covariance matrix. Therefore, Y_t is strictly stationary as well.

b. 1. We compare the simulated series with an independently generated $N(0, 1)$ white-noise sample. Both show irregular zero-centered fluctuations of similar size, with sample autocovariances near zero away from lag zero. For n consecutive observations the simulated Gaussian vector has covariance I_n , so it has exactly the same distribution as n independent standard normals. The infinite extension is periodic, $X_{t+n} = X_t$, and is not white noise at all lags.



2. We may write X_t explicitly as

$$X_t = \frac{1}{\sqrt{n}} \left(Z_0 + \sum_{j=1}^{n/2-1} (Z_{j1} \cos(\omega_j t) + Z_{j2} \sin(\omega_j t)) + Z_{n/2} (-1)^t \right)$$

and then

$$\begin{aligned} \gamma_X(t, t+h) &= \frac{1}{n} \left[1 + (-1)^h + 2 \sum_{j=1}^{n/2-1} \cos\left(\frac{2\pi j h}{n}\right) \right] \\ &= \begin{cases} 1, & h \in n\mathbb{Z}, \\ 0, & h \notin n\mathbb{Z}. \end{cases} \end{aligned}$$

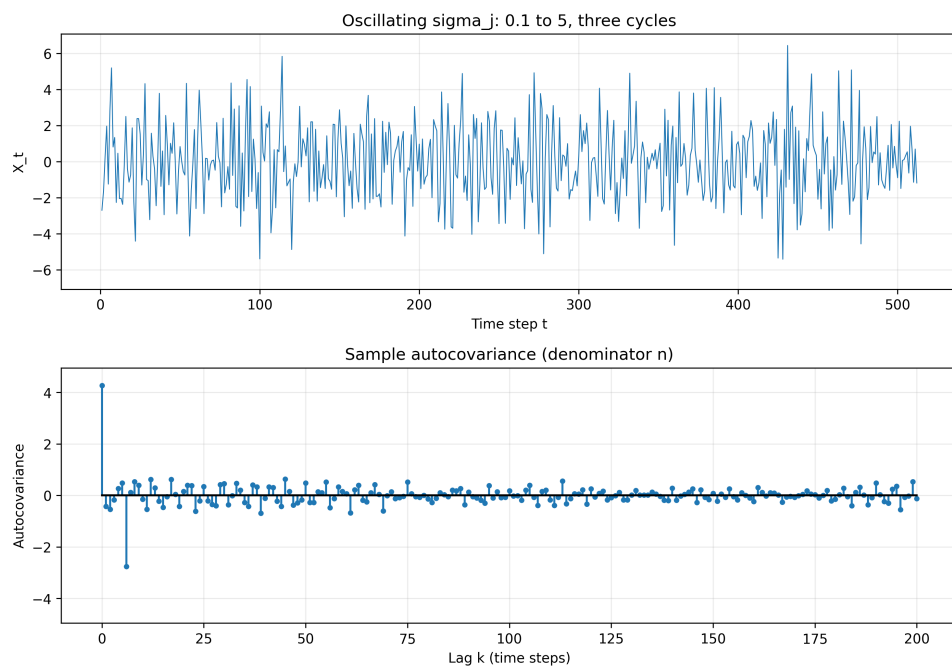
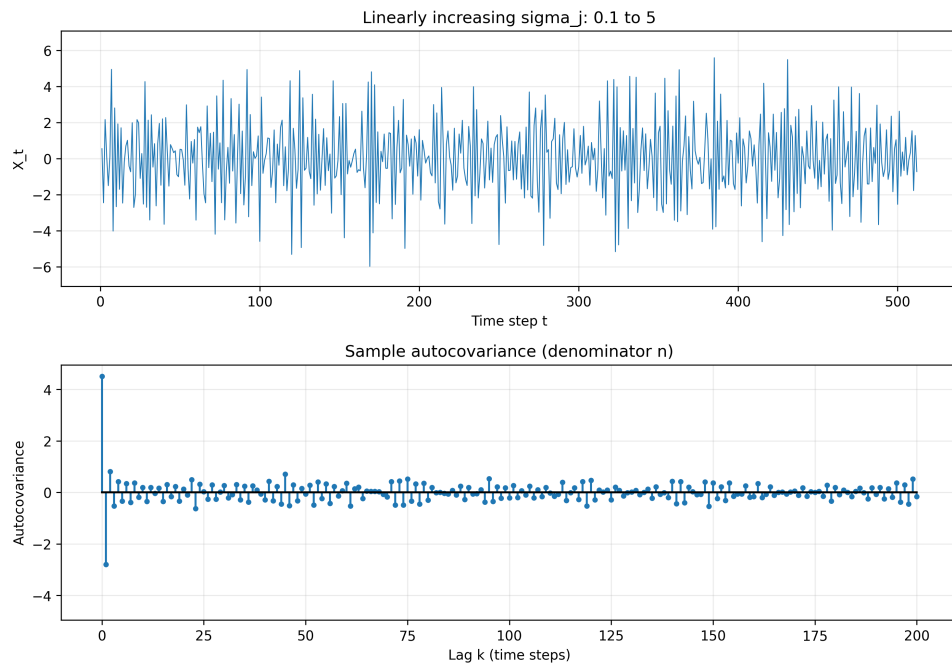
3. For $j = 1, \dots, 255$, define $u_j = (j-1)/254$. We compare

$$\sigma_j = 0.1 + 4.9u_j \quad \text{and} \quad \sigma_j = 0.1 + \frac{4.9}{2}(1 - \cos(6\pi u_j)).$$

In both cases, $\text{Var}(Z_{j1}) = \text{Var}(Z_{j2}) = \sigma_j^2$; the endpoint coefficients retain variance 1. The first choice increases linearly with frequency, whereas the second oscillates across frequency bands. Increasing σ_j emphasizes rapid oscillations and tends to produce negative short-lag covariance; decreasing weights would emphasize low frequencies and smoother variation. Oscillating weights emphasize selected frequency bands, creating a different lag-dependence pattern. The index j is frequency, not time: these weights do not make the variance increase with t . In fact,

$$\gamma_X(h) = \frac{1}{n} \left[1 + (-1)^h + \sum_{j=1}^{n/2-1} \sigma_j^2 \cos(2\pi j h/n) \right],$$

Both constructions are stationary Gaussian processes. Their covariance is generally nonzero at some nonzero lags.



A Python code

The scripts read `code/cet.csv` relative to their own location and save figures in `png/`. ACF uses the denominator n without variance normalization.

A.1 Sample autocovariance

```
1 import numpy as np
2
3
4 def sample_autocovariance(series, max_lag=None):
5
6     x = np.asarray(series, dtype=float)
7     if x.ndim != 1 or x.size == 0 or not np.isfinite(x).all():
8         raise ValueError("series must be a nonempty finite one-dimensional array")
9     n = len(x)
10    if max_lag is None:
11        max_lag = n - 1
12    if not isinstance(max_lag, (int, np.integer)) or not 0 <= max_lag < n:
13        raise ValueError("max_lag must be an integer between 0 and len(series)-1")
14    centered = x - x.mean()
15    return np.array([
16        np.dot(centered[k:], centered[:n - k]) / n
17        for k in range(max_lag + 1)
18    ])
```

A.2 CET data analysis

```
1 from pathlib import Path
2
3 import numpy as np
4 from autocovariance import sample_autocovariance
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import matplotlib.dates as mdates
8 from math import pi as pi
9
10 df = pd.read_csv(
11     Path(__file__).resolve().parent / "cet.csv",
12     parse_dates=["date"],
13     index_col="date"
14 )
15
16 df = df.sort_index()
17 if df.index.has_duplicates or df["temp"].isna().any():
18     raise ValueError("Duplicate dates or missing temperatures")
```

```

19 if not df.index.equals(pd.date_range(df.index[0], df.index[-1], freq="D")):
20     raise ValueError("The daily series contains gaps")
21
22 def Plot_and_meanvalue_check(
23     series, step, address, fig_l=10, fig_w=6, *,
24     times=None, title="Time series diagnostics", ylabel="Residual (deg C)",
25 ):
26
27
28     series = np.asarray(series, dtype=float)
29     if series.ndim != 1 or not np.isfinite(series).all():
30         raise ValueError("series must be a finite one-dimensional array")
31     if not isinstance(step, (int, np.integer)) or not 2 <= step <= len(series):
32         raise ValueError("step must be an integer between 2 and len(series)")
33     if times is None:
34         times = np.arange(1, len(series) + 1)
35     times = np.asarray(times)
36     if times.ndim != 1 or len(times) != len(series):
37         raise ValueError("times and series must have the same length")
38
39     rolling_mean = np.convolve(series, np.ones(step) / step, mode="valid")
40     rolling_std = pd.Series(series).rolling(step).std(ddof=1).to_numpy()[step - 1:]
41     window_times = times[step - 1:]
42
43     fig, axes = plt.subplots(2, 1, figsize=(fig_l, fig_w), sharex=True)
44     axes[0].plot(times, series, linewidth=0.6, alpha=0.6, label="Observations")
45     axes[0].plot(window_times, rolling_mean, color="tab:red",
46                 linewidth=1.5, label=f"{step}-observation rolling mean")
47     axes[0].axhline(series.mean(), color="0.4", linestyle="--", linewidth=0.8,
48                   label="Overall mean")
49     axes[0].set_title(title)
50     axes[0].set_ylabel(ylabel)
51     axes[0].legend(fontsize=8)
52     axes[1].plot(window_times, rolling_std, color="tab:orange",
53                label=f"{step}-observation rolling sample SD")
54     axes[1].set_ylabel("Standard deviation (deg C)")
55     axes[1].legend(fontsize=8)
56     if np.issubdtype(times.dtype, np.datetime64):
57         locator = mdates.AutoDateLocator()
58         axes[1].xaxis.set_major_locator(locator)
59         axes[1].xaxis.set_major_formatter(mdates.ConciseDateFormatter(locator))
60         axes[1].set_xlabel("Date")
61     else:
62         axes[1].set_xlabel("Day number")
63     for ax in axes:
64         ax.grid(alpha=0.25)
65     fig.tight_layout()

```

```

66     output = Path(address)
67     if not output.is_absolute():
68         output = Path(__file__).resolve().parents[1] / "png" / output
69     output.parent.mkdir(parents=True, exist_ok=True)
70     fig.savefig(output, dpi=300, bbox_inches="tight")
71     return fig, axes, rolling_mean, rolling_std
72
73
74 window_full = 365
75 fig, axes, mean_full, std_full = Plot_and_meanvalue_check(
76     df["temp"].to_numpy(), window_full, "cet_temp.png", 12, 6,
77     times=df.index,
78     title=f"Daily CET and {window_full}-day rolling statistics, 1772-2025",
79     ylabel="Daily mean temperature (deg C)",
80 )
81
82
83 x = df["temp"].iloc[:365].to_numpy()
84 x = x - np.mean(x)
85
86 n = len(x)
87 max_lag = 200
88
89
90 acf_values = sample_autocovariance(x, max_lag)
91
92 fig_2, ax_2 = plt.subplots(figsize=(10, 4))
93 ax_2.stem(np.arange(max_lag + 1), acf_values)
94 ax_2.set_xlabel("Lag (days)")
95 ax_2.set_ylabel("Autocovariance (deg C squared)")
96 ax_2.set_title("Sample autocovariance of daily CET for the first 365 days")
97 fig_2.tight_layout()
98 fig_2.savefig(
99     Path(__file__).resolve().parents[1] / "png" / "Lags.png",
100     dpi=300, bbox_inches="tight",
101 )
102
103
104 n = 365
105 x = df["temp"].iloc[:n].to_numpy()
106 t = np.arange(1, n + 1, dtype=float)
107 A = np.column_stack((np.ones(n), t))
108
109 beta, residuals, rank, singular_values = np.linalg.lstsq(A, x, rcond=None)
110
111 x_hat = A @ beta
112 resid = x - x_hat

```

```

113
114 resid_linear = resid.copy()
115 fig_3, axes_3, mean_linear, std_linear = Plot_and_meanvalue_check(
116     resid_linear, 30, "Residues.png", times=df.index[:n],
117     title="Linear-model residuals and 30-day rolling statistics",
118 )
119
120
121 n = 3*365
122
123 x = df["temp"].iloc[:n].to_numpy()
124
125 t = np.arange(1, n + 1, dtype=float)
126 angle_t = t*2*pi/365
127 A = np.column_stack((np.ones(n), t,np.cos(angle_t),np.sin(angle_t)))
128
129 beta, residuals, rank, singular_values = np.linalg.lstsq(A, x, rcond=None)
130
131 x_hat = A @ beta
132 resid = x - x_hat
133
134 resid_seasonal = resid.copy()
135 fig_4, axes_4, mean_seasonal, std_seasonal = Plot_and_meanvalue_check(
136     resid_seasonal, 30, "Tri_Residues.png", times=df.index[:n],
137     title="Seasonal-model residuals and 30-day rolling statistics",
138 )
139
140 for values, filename, title in [
141     (resid_linear, "Residual_ACov.png", "Linear-model residual autocovariance"),
142     (resid_seasonal, "Seasonal_ACov.png", "Seasonal-model residual autocovariance"),
143 ]:
144     covariance = sample_autocovariance(values, 200)
145     fig, ax = plt.subplots(figsize=(10, 4))
146     ax.stem(np.arange(201), covariance, markerfmt=".", basefmt="k-")
147     ax.set(xlabel="Lag (days)", ylabel="Autocovariance (deg C squared)", title=title)
148     ax.grid(alpha=0.25)
149     fig.tight_layout()
150     fig.savefig(Path(__file__).resolve().parents[1] / "png" / filename,
151                 dpi=300, bbox_inches="tight")
152     plt.close(fig)
153 plt.close("all")

```

A.3 Gaussian trigonometric simulations

```
1 from pathlib import Path
2
3 import numpy as np
4 from autocovariance import sample_autocovariance
5 import matplotlib.pyplot as plt
6
7
8 def generate_X(n=512, var_zj1=2.0, var_zj2=2.0, seed=None):
9
10
11     if isinstance(n, (bool, np.bool_)) or not isinstance(n, (int, np.integer)) or n < 2 or n %
12         2:
13         raise ValueError("n must be a positive even integer")
14
15     num_frequencies = n // 2 - 1
16
17     def variance_array(value, name):
18         value = np.asarray(value, dtype=float)
19         if not np.isfinite(value).all() or np.any(value < 0):
20             raise ValueError(f"{name} must contain finite, nonnegative variances")
21         if value.ndim == 0:
22             return np.full(num_frequencies, value.item())
23         if value.shape != (num_frequencies,):
24             raise ValueError(f"{name} must be a scalar or have shape ({num_frequencies},)")
25         return value
26
27     variance_1 = variance_array(var_zj1, "var_zj1")
28     variance_2 = variance_array(var_zj2, "var_zj2")
29     rng = np.random.default_rng(seed)
30     t = np.arange(1, n + 1)
31     X_components = np.zeros((n // 2 + 1, n))
32
33     Z_0, Z_half = rng.normal(0.0, 1.0, size=2)
34     X_components[0, :] = Z_0
35     X_components[n // 2, :] = Z_half * (-1.0) ** t
36
37     scales = np.sqrt(np.column_stack((variance_1, variance_2)))
38     Z = rng.normal(0.0, scales, size=(num_frequencies, 2))
39     for j in range(1, n // 2):
40         omega_j = 2 * np.pi * j / n
41         X_components[j, :] = (
42             Z[j - 1, 0] * np.cos(omega_j * t)
43             + Z[j - 1, 1] * np.sin(omega_j * t)
44         )
```

```

45
46     X = X_components.sum(axis=0) / np.sqrt(n)
47     return X
48
49
50 def plot_series_and_autocovariance(series, address, fig_l=10, fig_w=7, *,
51                                   max_lag=200, title="Simulated series",
52                                   ylim=None, series_ylim=None):
53
54     values = sample_autocovariance(series, max_lag)
55     lags = np.arange(len(values))
56     fig, axes = plt.subplots(2, 1, figsize=(fig_l, fig_w))
57     axes[0].plot(np.arange(1, len(series) + 1), series, linewidth=0.7)
58     axes[0].set(xlabel="Time step t", ylabel="X_t", title=title)
59     axes[1].stem(lags, values, markerfmt=".", basefmt="k-")
60     axes[1].set(xlabel="Lag k (time steps)", ylabel="Autocovariance",
61                title="Sample autocovariance (denominator n)")
62     if series_ylim is not None:
63         axes[0].set_ylim(series_ylim)
64     if ylim is not None:
65         axes[1].set_ylim(ylim)
66     for ax in axes:
67         ax.grid(alpha=0.25)
68     fig.tight_layout()
69     output = Path(address)
70     if not output.is_absolute():
71         output = Path(__file__).resolve().parents[1] / "png" / output
72     output.parent.mkdir(parents=True, exist_ok=True)
73     fig.savefig(output, dpi=300, bbox_inches="tight")
74     return fig, axes
75
76
77 if __name__ == "__main__":
78     n = 512
79     seed = 42
80     j = np.arange(1, n // 2)
81     u = (j - 1) / (len(j) - 1)
82     sigma_min, sigma_max = 0.1, 5.0
83     cycles = 3
84     sigma_linear = sigma_min + (sigma_max - sigma_min) * u
85     sigma_oscillating = sigma_min + (sigma_max - sigma_min) * (
86         1 - np.cos(2 * np.pi * cycles * u)
87     ) / 2
88     cases = [
89         ("sigma_linear", sigma_linear, "Linearly increasing sigma_j: 0.1 to 5"),
90         ("sigma_oscillating", sigma_oscillating, "Oscillating sigma_j: 0.1 to 5, three cycles")
91     ],

```

```

91 ]
92
93
94 series_by_case = {
95     name: generate_X(n, sigma**2, sigma**2, seed=seed)
96     for name, sigma, title in cases
97 }
98 max_lag = 200
99 series_limit = 1.1 * max(np.abs(x).max() for x in series_by_case.values())
100 limit = 1.1 * max(
101     np.abs(sample_autocovariance(x, max_lag)).max()
102     for x in series_by_case.values()
103 )
104 for name, sigma, title in cases:
105     fig, axes = plot_series_and_autocovariance(
106         series_by_case[name], f"HW1_5_{name}.png", 10, 7,
107         title=title,
108         max_lag=max_lag, ylim=(-limit, limit),
109         series_ylim=(-series_limit, series_limit),
110     )
111     plt.close(fig)
112
113 default = generate_X(n=n, seed=seed)
114 white_noise = np.random.default_rng(seed + 1).normal(size=n)
115 fig, axes = plt.subplots(2, 2, figsize=(12, 7), sharex="row", sharey="row")
116 for column, (values, title) in enumerate([
117     (default, "Trigonometric simulation, default variance"),
118     (white_noise, "Independent N(0,1) white noise"),
119 ]):
120     axes[0, column].plot(np.arange(1, n + 1), values, linewidth=0.7)
121     axes[0, column].set(title=title, xlabel="Time step t", ylabel="Value")
122     covariance = sample_autocovariance(values, max_lag)
123     axes[1, column].stem(np.arange(max_lag + 1), covariance,
124                          markerfmt=".", basefmt="k-")
125     axes[1, column].set(xlabel="Lag k", ylabel="Autocovariance")
126     for row in range(2):
127         axes[row, column].grid(alpha=0.25)
128 fig.tight_layout()
129 fig.savefig(Path(__file__).resolve().parents[1] / "png" / "HW1_5_default.png",
130             dpi=300, bbox_inches="tight")
131 plt.close(fig)

```